EXPERIMENT 1: Creating and Using a User-Defined Package

Question

Create a package named mypackage with a Student class that stores student details and displays them. Import and use Student from another class.

Program

Compile from the folder containing mypackage: javac mypackage/Student.java Main.java, then run: java Main.

mypackage/Student.java

```
package mypackage;
public class Student {
    public void display() {
        System.out.println("Name: Ravi");
        System.out.println("Roll No: 21");
    }
}
```

Main.java

```
import mypackage.Student;
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

Output

```
Name: Ravi
Roll No: 21
```

EXPERIMENT 2: Importing Packages

Question

Create package college with Student and Faculty classes. Show both a specific-class import and a wildcard import using college.*.

Program

The Main.java example uses college.*. To use a specific import instead, replace it with import college.Student; and import college.Faculty; if both classes are needed.

college/Student.java

```
package college;
public class Student {
    public void show() {
        System.out.println("Student class");
    }
```

```
}
```

college/Faculty.java

```
package college;
public class Faculty {
    public void show() {
        System.out.println("Faculty class");
    }
}
```

Main.java (wildcard import)

```
import college.*;
public class Main {
    public static void main(String[] args) {
        Student s = new Student();
        Faculty f = new Faculty();
        s.show();
        f.show();
    }
}
```

Alternative import for a specific class

```
import college.Student;
```

Output

```
Student class
Faculty class
```

EXPERIMENT 3: Packages and Member Access

Question

Create a class with public, private, protected and default members. Demonstrate access from the same class, another class in the same package, and a class in a different package.

Program

Run the class in pack1 to see the accessible members. The commented lines are intentionally not executed because they would cause compile errors.

pack1/AccessDemo.java

```
package pack1;
public class AccessDemo {
    public int a = 10;
    private int b = 20;
    protected int c = 30;
    int d = 40;

    public void showAll() {
        System.out.println(a + " " + b + " " + c + " " + d);
    }
}
```

pack1/SamePackage.java

```
package pack1;
public class SamePackage {
   public static void main(String[] args) {
      AccessDemo x = new AccessDemo();
      System.out.println(x.a); // public
      // x.b is not allowed: private
      System.out.println(x.c); // protected
      System.out.println(x.d); // default
   }
}
```

pack2/OtherPackage.java

```
package pack2;
import pack1.AccessDemo;
public class OtherPackage extends AccessDemo {
   public void show() {
      System.out.println(a); // public
      System.out.println(c); // protected through inheritance
      // b and d are not accessible here
   }
}
```

Output

```
AccessDemo.showAll(): 10 20 30 40
Same package: public, protected and default are accessible; private is not.
Different package subclass: public and protected are accessible; private and default are not.
```

EXPERIMENT 4: Demonstrate CLASSPATH

Question

Create a utilities package with Calculator, compile it, set CLASSPATH, and use Calculator from another program.

Program

From the project folder, compile: javac utilities/Calculator.java UseCalculator.java. Run: java UseCalculator. If classes are stored elsewhere, set CLASSPATH to the folder that contains the utilities folder (for example, set CLASSPATH=. on Windows Command Prompt, or export CLASSPATH=. on macOS/Linux).

utilities/Calculator.java

```
package utilities;
public class Calculator {
   public int add(int a, int b) {
      return a + b;
   }
}
```

UseCalculator.java

```
import utilities.Calculator;
public class UseCalculator {
   public static void main(String[] args) {
      Calculator c = new Calculator();
      System.out.println("Sum = " + c.add(5, 3));
   }
}
```

Output

```
Sum = 8
```

EXPERIMENT 5: Package Containing Multiple Classes

Question

Create a package named bank containing Account, Customer and Transaction classes. Access all three from a main class outside the package.

Program

Compile from the project folder: javac bank/*.java BankMain.java, then run: java BankMain.

bank/Account.java

```
package bank;
public class Account {
   public void show() { System.out.println("Account details"); }
}
```

bank/Customer.java

```
package bank;
public class Customer {
   public void show() { System.out.println("Customer details"); }
}
```

bank/Transaction.java

```
package bank;
public class Transaction {
   public void show() { System.out.println("Transaction details"); }
}
```

BankMain.java

```
import bank.*;
public class BankMain {
   public static void main(String[] args) {
      new Account().show();
      new Customer().show();
      new Transaction().show();
   }
}
```

Output

```
Account details
Customer details
```

Transaction details

EXPERIMENT 6: Defining and Implementing an Interface

Question

Create a Shape interface with area(). Implement it using Circle and Rectangle classes.

Program

ShapeDemo.java

```
interface Shape {
    double area();
}
class Circle implements Shape {
    public double area() { return 3.14 * 2 * 2; }
}
class Rectangle implements Shape {
    public double area() { return 5 * 4; }
}
public class ShapeDemo {
    public static void main(String[] args) {
        Shape c = new Circle();
        Shape r = new Rectangle();
        System.out.println("Circle area: " + c.area());
        System.out.println("Rectangle area: " + r.area());
    }
}
```

Output

```
Circle area: 12.56
Rectangle area: 20.0
```

EXPERIMENT 7: Implementing Multiple Interfaces

Question

Create Printable and Showable interfaces with print() and show(). Implement both in a Demo class.

Program

Demo.java

```
interface Printable {
    void print();
}
interface Showable {
    void show();
}
public class Demo implements Printable, Showable {
    public void print() {
        System.out.println("Printing...");
    }
```

```
    public void show() {
        System.out.println("Showing...");
    }
    public static void main(String[] args) {
        Demo d = new Demo();
        d.print();
        d.show();
    }
}
```

Output

```
Printing...
Showing...
```

EXPERIMENT 8: Interface-Based Polymorphism

Question

Create a Vehicle interface with start(). Implement it using Car and Bike, and assign both objects to a Vehicle reference.

Program

VehicleDemo.java

```
interface Vehicle {
    void start();
}
class Car implements Vehicle {
    public void start() { System.out.println("Car starts"); }
}
class Bike implements Vehicle {
    public void start() { System.out.println("Bike starts"); }
}
public class VehicleDemo {
    public static void main(String[] args) {
        Vehicle v = new Car();
        v.start();
        v = new Bike();
        v.start();
    }
}
```

Output

```
Car starts
Bike starts
```

EXPERIMENT 9: Variables in Interfaces

Question

Create an interface Constants with MAX_MARKS and PI. Access the constants using the interface name and show that they cannot be changed.

Program

Interface variables are public, static and final by default. The commented assignment would cause a compile-time error.

ConstantsDemo.java

```
interface Constants {
    int MAX_MARKS = 100;
    double PI = 3.14159;
}
public class ConstantsDemo {
    public static void main(String[] args) {
        System.out.println("Maximum marks: " + Constants.MAX_MARKS);
        System.out.println("PI: " + Constants.PI);
        // Constants.MAX_MARKS = 50; // Error: constant
    }
}
```

Output

```
Maximum marks: 100
PI: 3.14159
```

EXPERIMENT 10: Nested Interfaces

Question

Create an outer class University containing a nested interface Department. Implement the nested interface in another class.

Program

UniversityDemo.java

```
class University {
    interface Department {
        void display();
    }
}
class CSE implements University.Department {
    public void display() {
        System.out.println("Computer Science Department");
    }
}
public class UniversityDemo {
    public static void main(String[] args) {
        University.Department d = new CSE();
```

```
        d.display();
    }
}
```

Output

```
Computer Science Department
```

EXPERIMENT 11: Interface Inheritance

Question

Create interface Animal with eat(), interface Dog extending Animal with bark(), and implement both methods in Labrador.

Program

LabradorDemo.java

```
interface Animal {
    void eat();
}
interface Dog extends Animal {
    void bark();
}
public class LabradorDemo implements Dog {
    public void eat() {
        System.out.println("Labrador eats");
    }
    public void bark() {
        System.out.println("Labrador barks");
    }
    public static void main(String[] args) {
        LabradorDemo d = new LabradorDemo();
        d.eat();
        d.bark();
    }
}
```

Output

```
Labrador eats
Labrador barks
```

EXPERIMENT 12: Real-World Application Using Interfaces

Question

Create a payment system using a Payment interface with pay(double amount). Implement it using CreditCardPayment, UPIPayment and NetBankingPayment. Let the user select a method and process the payment.

Program

Sample output shown for option 2 (UPI) and amount 500. The program also accepts options 1 and 3.

PaymentDemo.java

```
import java.util.Scanner;
interface Payment {
   void pay(double amount);
}
class CreditCardPayment implements Payment {
   public void pay(double amount) {
      System.out.println("Paid by Credit Card: " + amount);
   }
}
class UPIPayment implements Payment {
   public void pay(double amount) {
      System.out.println("Paid by UPI: " + amount);
   }
}
class NetBankingPayment implements Payment {
   public void pay(double amount) {
      System.out.println("Paid by Net Banking: " + amount);
   }
}
public class PaymentDemo {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.println("1. Credit Card  2. UPI  3. Net Banking");
      System.out.print("Choose method: ");
      int choice = sc.nextInt();
      System.out.print("Enter amount: ");
      double amount = sc.nextDouble();

      Payment p;
      if (choice == 1) p = new CreditCardPayment();
      else if (choice == 2) p = new UPIPayment();
      else if (choice == 3) p = new NetBankingPayment();
      else {
         System.out.println("Invalid choice");
         sc.close();
         return;
      }
      p.pay(amount);
      sc.close();
   }
}
```

Output

```
1. Credit Card  2. UPI  3. Net Banking
Choose method: 2
Enter amount: 500
Paid by UPI: 500.0
```